NUST CHIP DESIGN CENTRE

Physical Design

IO Pads Placement

Name	***Sidra Naveed***
Instructor	*Hira Sohail*
Date	*13th October 2025*

Introduction:

In modern integrated circuit (IC) design, physical implementation is a critical stage where the abstract representation of a design is transformed into a manufacturable layout. One of the most important steps in this process is the placement of Input/Output (IO) pads. IO pads serve as the physical interface between the internal core logic of the chip and the external environment, enabling communication with other chips, packages, and the printed circuit board (PCB).

Proper IO pad placement is essential for:

- Ensuring signal integrity by minimizing noise and cross-talk.
- Supporting reliable power delivery through dedicated VDD and VSS pads distributed around the periphery. Meeting electrostatic discharge (ESD) protection requirements.
- Achieving correct bonding or bump connections in packaging.
- Maintaining an optimized floorplan that balances core utilization, power routing, and timing closure.

This lab focuses on learning the principles and methodology of IO pad placement in ASIC physical design.

Through hands-on steps, we will explore how to:

- Define the die and core area.
- Allocate space for power rings and routing margins.
- Place IO pads systematically around the die.
- Connect IO pads to the core logic for simulation and physical verification.

Objective:

The objective of this lab is to Place IO Standard Cells of desired library in our desired RTL design.

- Understand the IO placement locations in IO file.
- Pre/Post Simulate the IO Pads based RTL Design to verify the functionality.
- Final STA after layout to verify whether there is timing violation or not.

Theory of Concept:

When an IC is manufactured, it needs a way to communicate with the outside world for example, to send or receive signals from other chips or to connect to power. However, the logic inside a chip works at small voltage levels and cannot be connected directly to external devices. This is where IO (Input/Output) pads come in.

IO pads are special buffer cells placed around the boundary of a chip. They act as an interface between the internal core logic and the external world. These pads protect the core from high voltages, provide proper signal driving strength, and help with power distribution.

In the physical design flow, after the core is synthesized and before the layout begins, the IO pads must be placed properly on the periphery of the chip. This is known as IO pad placement. The placement of pads is very important because it directly affects routing, timing, and how power is supplied across the chip.

There are different types of pads used in a design:

- Input pads – receive signals from the external environment and send them into the core.
- Output pads – drive signals from the core to the outside world.
- Power (VDD) and Ground (VSS) pads – provide power to the chip and distribute it evenly
- Corner pads – used at each corner to complete the pad ring and for chip connectivity.
- Clock and Reset pads – handle special signals that control the chip operation.

In this lab, we connect the 8-bit ALU design with its corresponding IO pad cells using a Top Wrapper module. Each input, output, clock, and reset signal from the ALU is connected to an appropriate pad cell. The pads are then arranged around the die boundary according to the IO file in the Innovus tool.

The IO file defines how and where each pad should be placed (top, bottom, left, or right side). Once the pads are placed, power rings and power routing are created around the pads to distribute power evenly to the entire chip.

Finally, the design goes through the standard steps of the physical design process — synthesis, floorplanning, placement, clock tree synthesis (CTS), routing, and verification — but now the design includes both the core logic and the IO pads. This results in a complete and ready-to-fabricate chip layout.

Front End:

RTL code of ALU:

```
`timescale 1ns/10ps
module ALU(Out, A, B, Sel, clk, rst);
    input [7:0] A, B;
    input [1:0] Sel;
    input clk, rst;
    output reg [7:0] Out;

    wire [7:0] M1, A1, N1, N2;
    reg  [7:0] M_Out;

    assign M1 = A & B;    // AND
    assign A1 = A | B;    // OR
    assign N1 = ~A;       // NOT A
    assign N2 = ~B;       // NOT B

    always @(posedge clk) begin
        if (!rst)
            Out <= 8'b0;
        else
            Out <= M_Out;
    end

    always @(*) begin
        case (Sel)
            2'b00 : M_Out = M1;
            2'b01 : M_Out = A1;
            2'b10 : M_Out = N1;
            2'b11 : M_Out = N2;
        endcase
    end
endmodule
```

Explanation:

This ALU (Arithmetic Logic Unit) RTL design performs **basic logical operations** between two 8-bit inputs, **A** and **B**, depending on a 2-bit selection input (**Sel**). It also includes a **clock** and **reset** signal for synchronous operation.

Here's how it works in simple terms:

- The inputs **A** and **B** are both 8-bit numbers.
- The **Sel** signal decides which operation the ALU will perform:
    - 00 → performs **AND** operation (A & B)
    - 01 → performs **OR** operation (A | B)
    - 10 → performs **NOT A** operation (~A)
    - 11 → performs **NOT B** operation (~B)

These operations are done using assign statements that generate four intermediate results: **M1**, **A1**, **N1**, and **N2**.

The selected result is stored in a temporary register **M_Out** based on the value of **Sel**. On every **positive clock edge**, the final output **Out** is updated with **M_Out**. If the **reset (rst)** signal is low (0), the output **Out** becomes 0 (cleared).

In short, this ALU takes two inputs, performs one of four logic operations selected by **Sel**, and gives the result at the output **Out** on every clock cycle — unless reset is active.

Wrapper File

```
`timescale 1ns/10ps
module top_wrapper (
    input  [7:0] A, B,
    input  [1:0] Sel,
    input        clk, rst,
    output [7:0] Out
);

    // Internal nets (core side)
    wire [7:0] wA, wB, wOut;
    wire [1:0] wSel;
    wire       wclk, wrst;

    // Core logic instantiation
    ALU uut (
        .A   (wA),
        .B   (wB),
        .Sel (wSel),
        .clk (wclk),
        .rst (wrst),
        .Out (wOut)
    );


    // I/O PAD INSTANTIATIONS

    // Clock Pad
    PXOE3CDG clk_pad (
        .E   (1'b1),
        .XIN (clk),
        .XOUT(),
        .XC  (wclk)
    );

    // Reset Pad (input)
    PDDW16DGZ rst_inst (
        .OEN (1'b1),
        .I   (1'b1),
        .PAD (rst),
        .REN (1'b1),
        .C   (wrst)
    );
```

Explanation of Top Wrapper Module
The top_wrapper module is the top-level design that connects the ALU core logic with the IO pad cells**.** In VLSI design, pad cells act as the physical interface between the internal circuit (core) and the external world (like PCB pins or package connections). The purpose of this wrapper is to **link the** ALU inputs and outputs through proper IO pads for simulation and physical implementation.

Structure Overview

The module has:

- **Inputs:** A[7:0], B[7:0], Sel[1:0], clk, rst
- **Output:** Out[7:0]

Inside the module, there are **internal wires** (wA, wB, wSel, wclk, wrst, wOut) that connect the pad outputs to the ALU inputs and vice versa.

1. Core Logic Connection

The **ALU** module is instantiated as uut (unit under test).
It performs logical operations (AND, OR, NOT) based on the control signal Sel.
The ALU inputs and outputs are connected to internal wires (wA, wB, wSel, wclk, wrst, wOut) which are then linked to external pins through IO pads.

2. IO Pad Cells
Each input/output of the ALU is connected to an IO pad cell. These pads handle signal buffering, protection (like ESD), and connection to the outside environment.
There are two main pad types used:

- PXOE3CDG → Used for the **clock input**
- PDDW16DGZ → Used for **reset, data inputs (A, B, Sel)** and **outputs (Out)**

3. Clock Pad
PXOE3CDG clk_pad (.E(1'b1), .XIN(clk), .XOUT(), .XC(wclk));

- This pad takes the external clock clk and sends it to the ALU as wclk.
- E=1 enables the pad.
- XOUT is unused since we only need the clock to go inside the chip.

4. Reset Pad
PDDW16DGZ rst_inst (.OEN(1'b1), .I(1'b1), .PAD(rst), .REN(1'b1), .C(wrst));

- The reset signal from outside (rst) is fed into the pad.
- The output of the pad (C) is sent inside as wrst.
- OEN=1 disables output (so it works as input).
- REN=1 enables input functionality.

5. Input Pads for A, B, and Sel
Each bit of **A**, **B**, and **Sel** is connected to an input pad:
PDDW16DGZ A_inst0 (.OEN(1'b1), .I(1'b1), .PAD(A[0]), .REN(1'b1), .C(wA[0]));

- These pads take external inputs (from the chip pins) and send them to the ALU.

- All inputs have OEN=1 (output disabled) and REN=1 (input enabled).

So, all signals like A[7:0], B[7:0], and Sel[1:0] go through these pads before reaching the ALU.

6. Output Pads

Each output bit of the ALU (wOut) goes through an output pad:

PDDW16DGZ out_inst0 (.OEN(1'b0), .I(wOut[0]), .PAD(Out[0]), .REN(), .C());

- These pads send data **from the ALU to the external pins**.
- OEN=0 enables output mode.
- REN and C are not needed because the pad is used as an output.

This setup ensures the ALU's internal output signals are properly driven to the outside world.

7. Working Summary

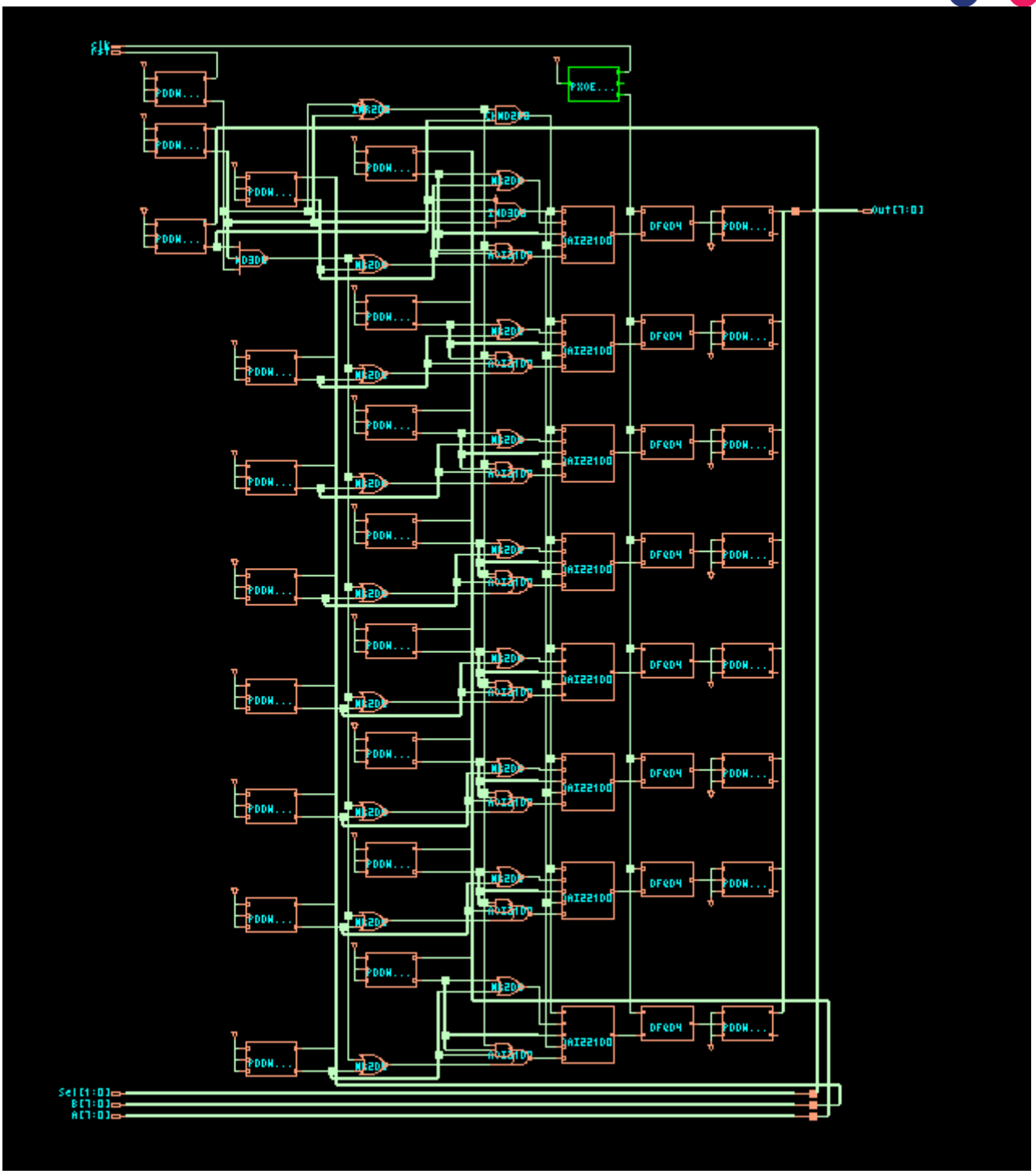
1. External inputs (A, B, Sel, clk, rst) come through the **input pads**.
2. The pad cells clean, protect, and buffer the signals before passing them to the ALU.
3. The ALU processes the data according to the selected operation.
4. The results go back out through **output pads** to the external output pins.

8. Purpose of the Wrapper

The **top_wrapper** acts as the **interface between the ALU and the physical chip environment**. This wrapper design is an essential step before **floorplanning and pad placement** in the physical design process.

Running Script file:

Netlist:


Out[7:0]
Sel[1:0]
B[7:0]
A[7:0]

Reports generated after synthesis:

Area Report:

top_wrapper_area.rpt

home > cc > Downloads > Lab_04_Materials > ALU_8B_TPZN > RPT > top_wrapper_area.rpt

```
1  ============================================================
2    Generated by:           Genus(TM) Synthesis Solution 21.18-s082_1
3    Generated on:           Oct 08 2025  05:22:31 pm
4    Module:                 top_wrapper
5    Operating conditions:   BCCOM (balanced_tree)
6    Wireload mode:          segmented
7    Area mode:              timing library
8  ============================================================
9
10   Instance  Module  Cell Count  Cell Area  Net Area   Total Area      Wireload
11 -----------------------------------------------------------------------------------
12 top_wrapper                71 154040.760     0.000   154040.760 ZeroWireload (S)
13   (S) = wireload was automatically selected
14
```

Explanation:

This area report shows the results of synthesis for the top_wrapper module. It tells us how much chip area the design will use after converting the RTL into gate-level cells. The report shows that the design contains 71 standard cells**,** with a total cell area of about 154,040.76 units. The wireload model used is segmented, which estimates the interconnect lengths for timing and area calculation. The report also mentions that the ZeroWireload model was automatically selected, meaning wire delays were assumed to be minimal or ideal. In simple terms, this report summarizes how much physical area the synthesized ALU design will occupy on the chip.

Gates Report:

```
============================================================
  Generated by:           Genus(TM) Synthesis Solution 21.18-s082_1
  Generated on:           Oct 08 2025  05:22:31 pm
  Module:                 top_wrapper
  Operating conditions:   BCCOM (balanced_tree)
  Wireload mode:          segmented
  Area mode:              timing library
============================================================
```

Gate	Instances	Area	Library
AOI31D0	8	20.160	tcbn65lpbc
CKND2D0	1	1.440	tcbn65lpbc
DFQD4	8	66.240	tcbn65lpbc
IND3D0	1	2.520	tcbn65lpbc
INR2D0	1	2.160	tcbn65lpbc
ND3D0	1	2.160	tcbn65lpbc
NR2D0	16	23.040	tcbn65lpbc
OAI221D0	8	23.040	tcbn65lpbc
PDDW16DGZ	27	153900.000	tpzn65lpgv2od3bc
total	71	154040.760	

Library	Instances	Area	Instances %
tcbn65lpbc	44	140.760	62.0
tpzn65lpgv2od3bc	27	153900.000	38.0

Type	Instances	Area	Area %
timing_model	27	153900.000	99.9
sequential	8	66.240	0.0
unresolved	1	0.000	0.0
logic	36	74.520	0.0
physical_cells	0	0.000	0.0
total	72	154040.760	100.0

Explanation:

This gate report shows the detailed breakdown of all the standard cells used in the top_wrapper design after synthesis. It lists how many instances of each type of gate were used, their individual and total areas, and which library they came from. Most of the area (about 99.9%**)** comes from I/O pad cells (PDDW16DGZ)**,** which occupy around 153,900 units of area, while the actual logic and flip-flops (like AOI, OAI, and DFQ cells) take up a very small portion. This means that the pad cells dominate the total chip area, since they are physically large compared to logic gates. The design uses two libraries — tcbn65lpbc for logic and flip-flops, and tpzn65lpgv2od3bc for pad cells. In short, this report shows that the majority of the chip area is used for input/output pads rather than the core logic of the ALU.

Power Report:

```
Instance: /top_wrapper
Power Unit: W
PDB Frames: /stim#0/frame#0
  ------------------------------------------------------------------------
  | Category         Leakage     Internal    Switching        Total    Row% |
  ------------------------------------------------------------------------
  |   memory     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00% |
  |  register    3.50477e-08  2.65998e-06  8.37049e-06  1.10655e-05   0.03% |
  |    latch     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00% |
  |    logic     1.45186e-08  1.27310e-06  7.85822e-07  2.07344e-06   0.01% |
  |     bbox     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00% |
  |    clock     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00% |
  |      pad     2.16653e-07  9.47588e-03  2.74726e-02  3.69487e-02  99.96% |
  |       pm     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00% |
  ------------------------------------------------------------------------
  | Subtotal     2.66220e-07  9.47982e-03  2.74818e-02  3.69619e-02 100.00% |
  |Percentage           0.00%       25.65%       74.35%      100.00% 100.00% |
  ------------------------------------------------------------------------
```

Explanation:

This **power report** shows how much power each part of the top_wrapper design consumes after synthesis. The total power is around 0.0369 W (36.9 mW)**.** Most of the power (about 99.96%**)** comes from the pad cells**,** which handle input and output signals and therefore have higher switching and internal power. The **logic and register** parts consume very little power (less than 0.05%), showing that the **core ALU logic is quite power efficient**. The report also shows that **switching power (74%)** is the main contributor, meaning most power is used when signals toggle during operation. Overall, this result indicates that while the internal logic consumes very little power, the I/O pads dominate the total power usage of the design.

qor report:

```
Timing
-------
  Cost    Critical         No of gates on  Violating
 Group   Path Slack  TNS    Critical Path    Paths
--------------------------------------------------
default    No paths   0.0
--------------------------------------------------
Total                 0.0                        0

Instance Count
--------------
Leaf Instance Count              71
Physical Instance count           0
Sequential Instance Count         8
Combinational Instance Count     63
Hierarchical Instance Count       1

Area
----
Cell Area                              154040.760
Physical Cell Area                     0.000
Total Cell Area (Cell+Physical)        154040.760
Net Area                               0.000
Total Area (Cell+Physical+Net)         154040.760

Max Fanout                             58 (n_36)
Min Fanout                             0 (Out[7])
Average Fanout                         2.3
Terms to net ratio                     3.2872
Terms to instance ratio                4.3521
Runtime                                8.873987 seconds
Elapsed Runtime                        10 seconds
Genus peak memory usage                1822.00
Innovus peak memory usage              no_value
Hostname                               ncdc-0053
```

Explanation:

This QoR (Quality of Results) report shows the overall performance summary of the synthesized top_wrapper design. It indicates that there are no timing violations, meaning the design meets all timing constraints successfully. The report shows a total of 71 logic cells**,** out of which 8 are sequential (flip-flops**)** and 63 are combinational gates. The total cell area is around 154,040.76 units, matching the area report. The maximum fanout (number of outputs driven by a single gate) is 58, and the average fanout is 2.3, which means signal loading is within a normal range. The synthesis process completed in about 10 seconds, showing that the design is small and efficiently synthesized without any timing or area issues.

I/O file for Innovus

```
(globals
    version = 3
    io_order = clockwise
    space = 10
)

(iopad
    ;====================== Corner Pads ======================
    (topleft
        (inst name="CornerCell1" cell=PCORNER orientation=R270 place_status=fixed)
    )
    (topright
        (inst name="CornerCell2" cell=PCORNER orientation=R180 place_status=fixed)
    )
    (bottomright
        (inst name="CornerCell3" cell=PCORNER orientation=R90 place_status=fixed)
    )
    (bottomleft
        (inst name="CornerCell4" cell=PCORNER orientation=R0 place_status=fixed)
    )

    ;====================== Top Edge ======================
    ; B[7:0] + Sel[1:0]
    (top
        (inst name="B_inst0" cell=PDDW16DGZ place_status=fixed)
        (inst name="B_inst1" cell=PDDW16DGZ place_status=fixed)
        (inst name="B_inst2" cell=PDDW16DGZ place_status=fixed)
        (inst name="PVDDR" cell=PVDD1DGZ place_status=fixed)
        (inst name="B_inst3" cell=PDDW16DGZ place_status=fixed)
        (inst name="B_inst4" cell=PDDW16DGZ place_status=fixed)
```

Explanation:

This I/O file defines the pad placement and configuration for the synthesized top-level module (top_wrapper) used during physical design in Cadence Innovus. It specifies how all input/output pads, power pads, and corner pads are arranged around the die boundary in a clockwise order**.** Each section — Top, Bottom, Left, and Right — lists instances of pads such as PDDW16DGZ (data and control I/O pads), PVDD1DGZ and PVSS1DGZ (power and ground pads), and PXOE3CDG (clock pad). The four corners use PCORNER cells to define the chip's physical corners. Every pad instance includes attributes like its name**,** cell type**,** orientation, and placement status (fixed). This file is essential for guiding the place-and-route tool to correctly align the pad ring, ensure proper power distribution, and maintain consistent I/O ordering around the chip per the design specifications

Post Synthesis:

Commands:

csh

Source cshrc

xrun -timescale 1ns/10ps /home/cc/Downloads/Lab_04_Materials/ALU_8B_TPZN/Netlist/top_wrapper_map.v /home/cc/Downloads/Lab_04_Materials/ALU_8B_TPZN/tb_top_wrapper.sv -v ALU.v tpzn65lpgv2od3.v -access +rwc -mess -gui

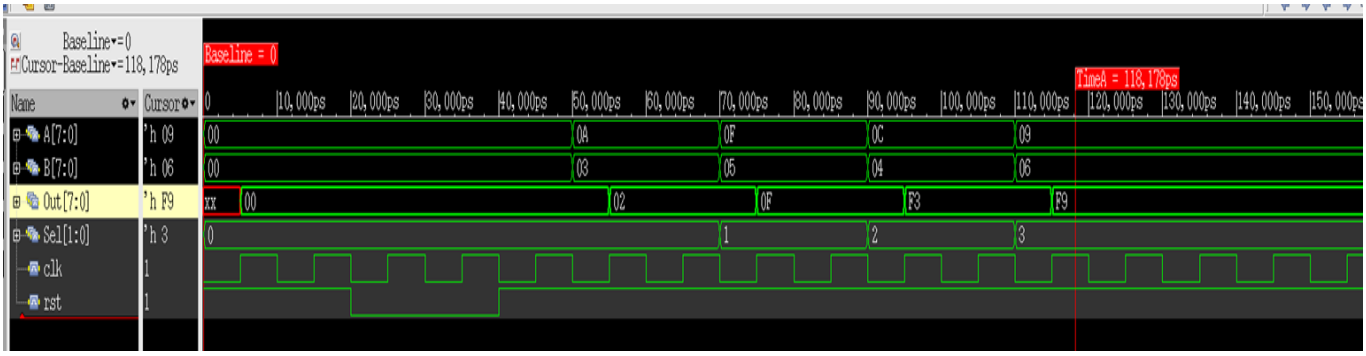
Terminal Output:

```
Design hierarchy summary:
                         Instances  Unique
        Modules:               203     143
        UDPs:                   40       2
        Primitives:            602      10
        Timing outputs:        304     111
        Registers:             197     139
        Scalar wires:          839       -
        Expanded wires:         18       3
        Always blocks:         127     101
        Initial blocks:          2       2
        Pseudo assignments:     98       -
        Timing checks:          80       -
        Process Clocks:        126     126
        Simulation timescale:  1ps
Writing initial simulation snapshot: worklib.tb_top_wrapper:sv
std          /STD
```

Comment:

This terminal output shows the design hierarchy summary generated during simulation. It confirms that all modules, registers, and connections in the ALU design were successfully compiled. The report lists the total instances, primitives, timing outputs, registers, and process clocks used in the design. The message “Writing initial simulation snapshot” means the simulator created a compiled version of the testbench for execution.

Waveform:



Constraint file (.sdc)

Clock Definition

```
create clock -name CLK -period 10 -waveform {0 5} [get pins {clk inst/XC}]
```

In previous labs, you have used **get_ports** that refers to top-level design ports (chip- level inputs/outputs) but in this case you should use **get_pins** that refers to internal pins of cells/instances inside the netlist. We written as *get_pins {clk_inst/XC}* where clk_inst is instance name (written in RTL) and XC is output that moves towards the core.

Input Constraints

```
set_input_delay -clock CLK -max 2.0 [get_pins {A_inst0/C A_inst1/C A_inst2/C A_inst3/C A_inst4/C A_inst5/C A_inst6/C A_inst7/C}]
set_input_delay -clock CLK -min 0.5 [get_pins {A_inst0/C A_inst1/C A_inst2/C A_inst3/C A_inst4/C A_inst5/C A_inst6/C A_inst7/C}]
```

In this statement, we use *get_pins {A_inst[*]/C}* where A_inst[*]. is instance name and C is output that moves towards core.

Output Constraints

```
set_output_delay -clock CLK -max 5.0 [get_pins {out_inst0/I out_inst1/I out_inst2/I out_inst3/I out_inst4/I out_inst5/I out_inst6/I out_inst7/I}]
set_output_delay -clock CLK -min 2.0 [get_pins {out_inst0/I out_inst1/I out_inst2/I out_inst3/I out_inst4/I out_inst5/I out_inst6/I out_inst7/I}]
```

In this statement, we use *get_pins {out_inst [*]/I}* where out_inst [*] is instance name and I is input that takes O/P from core.

Backend of Lab:

Synthesis

I will use 8-Bit ALU for front-end & back-end process of Digital IC Design. For this purpose, I will use Genus to synthesize & obtain technology mapped gate-level netlist. To initiate this process, we need the following files which is given below:

- script.tcl (tool command language that will guide the Genus to run desired design)
- constraint.sdc (defines how the tool should understand timing, clocks, IO delays, loads, and uncertainties)
- tpzn65lpgv2od3tc.lib
- tcbn65lptc.lib
- ALU_8B.v (Main RTL Design)
- Top_Wrapper.sv (Instantiation with RTL design & I/O Pads Cells)

After synthesizing, I will get netlist of your ALU design. Also I perform post simulate your synthesized design to check functionality. Now we move towards the back-end process of DIC Design.

Placement & Route

Top_wrapper.view:

```
# These timing libraries are provided by the foundry and must match the design's PVT conditions.
# Make sure that your lib path is different in other PC's
create_library_set -name TC_LIBSET -timing /home/cc/Downloads/Lab_04_Materials/ALU_8B_TPZN/mmmc_files/libs/tcbn65lptc.lib
create_library_set -name WC_LIBSET -timing /home/cc/Downloads/Lab_04_Materials/ALU_8B_TPZN/mmmc_files/libs/tcbn65lpwc.lib
create_library_set -name BC_LIBSET -timing /home/cc/Downloads/Lab_04_Materials/ALU_8B_TPZN/mmmc_files/libs/tcbn65lpbc.lib
```

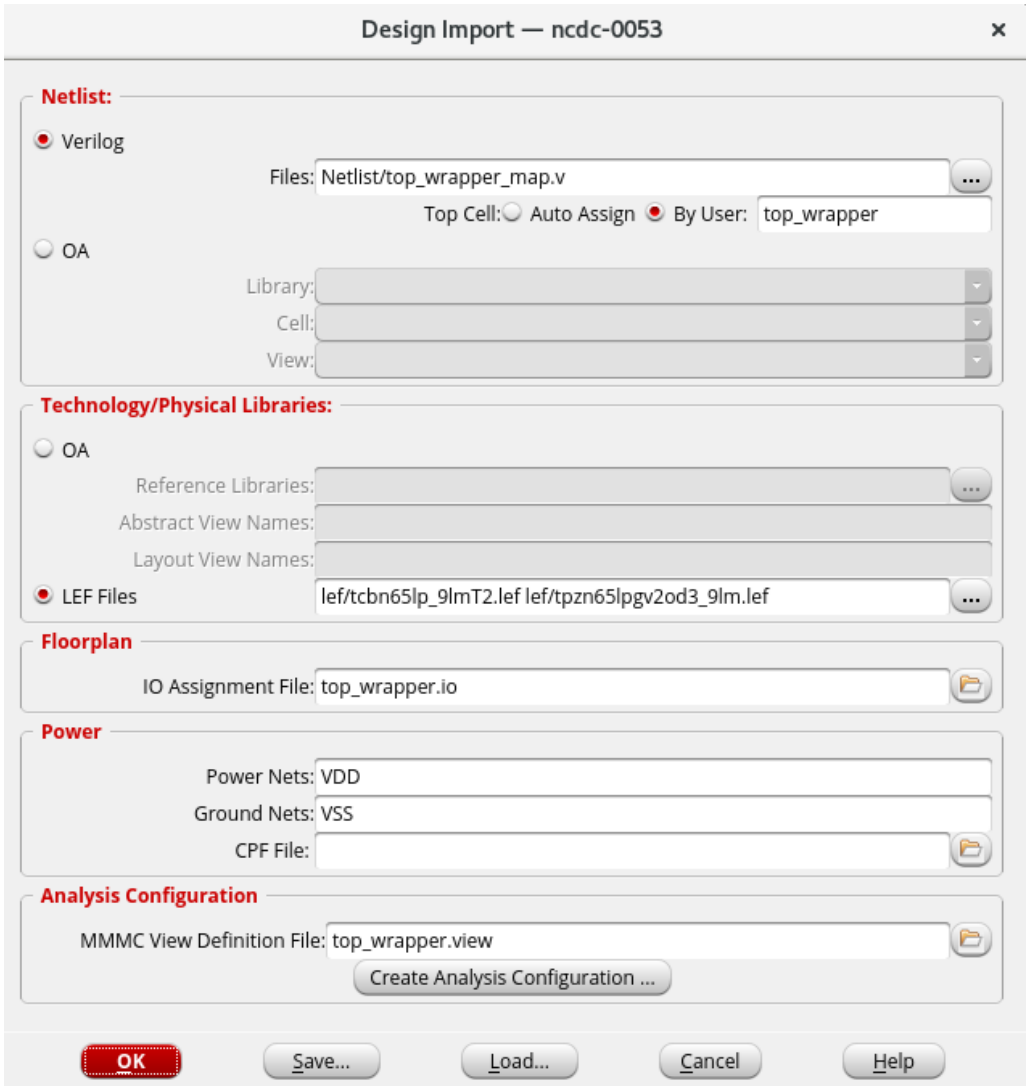
```
create_rc_corner -name RC_TC -cap_table /home/cc/Downloads/Lab_04_Materials/ALU_8B_TPZN/mmmc_files/captable/cln65lp_1p09m+alrdl_top2_typical.captable
create_rc_corner -name RC_WC -cap_table /home/cc/Downloads/Lab_04_Materials/ALU_8B_TPZN/mmmc_files/captable/cln65lp_1p09m+alrdl_top2_rcworst.captable
create_rc_corner -name RC_BC -cap_table /home/cc/Downloads/Lab_04_Materials/ALU_8B_TPZN/mmmc_files/captable/cln65lp_1p09m+alrdl_top2_rcbest.captable
```

```
# Make sure about the path of sdc
create_constraint_mode -name constraints -sdc_files {/home/cc/Downloads/Lab_04_Materials/ALU_8B_TPZN/constraint-ioPad.sdc}
```

Comment:

Here I changed the paths of the files needed in top wrapper view.

Initializing the Design



After initializing the design, the view looks like this.



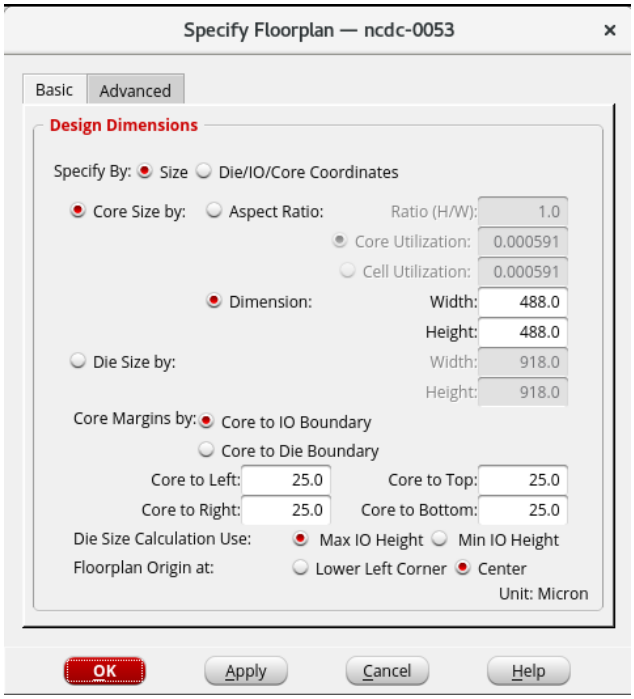
I have observed that the I/O pads are placed according to the I/O file that describes the coordinates of each cell.

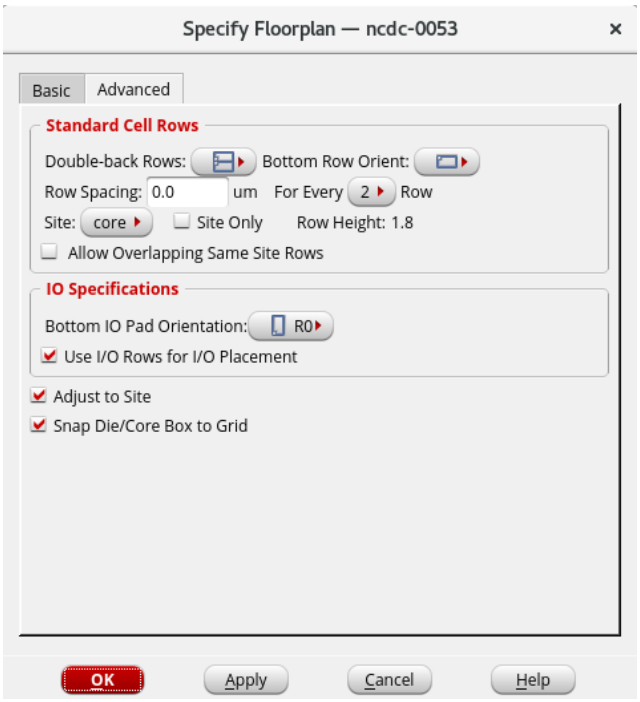
Floor planning:

It is a critical stage in the physical design flow where you define the physical organization and layout of the entire chip before placement. It sets the blueprint for how standard cells, macros, IO pads, and power structures will be arranged on the silicon die. It defines:

- Core area
- Die area
- Macros/IPs
- IO pins
- Core utilization
- Aspect ratio

For this I read my area report . During Front end my core was 166382.120µm2





My Area Report:

```
====================================================================
  Generated by:           Genus(TM) Synthesis Solution 21.18-s082_1
  Generated on:           Oct 08 2025  07:30:34 pm
  Module:                 top_wrapper
  Operating conditions:   BCCOM (balanced_tree)
  Wireload mode:          segmented
  Area mode:              timing library
====================================================================

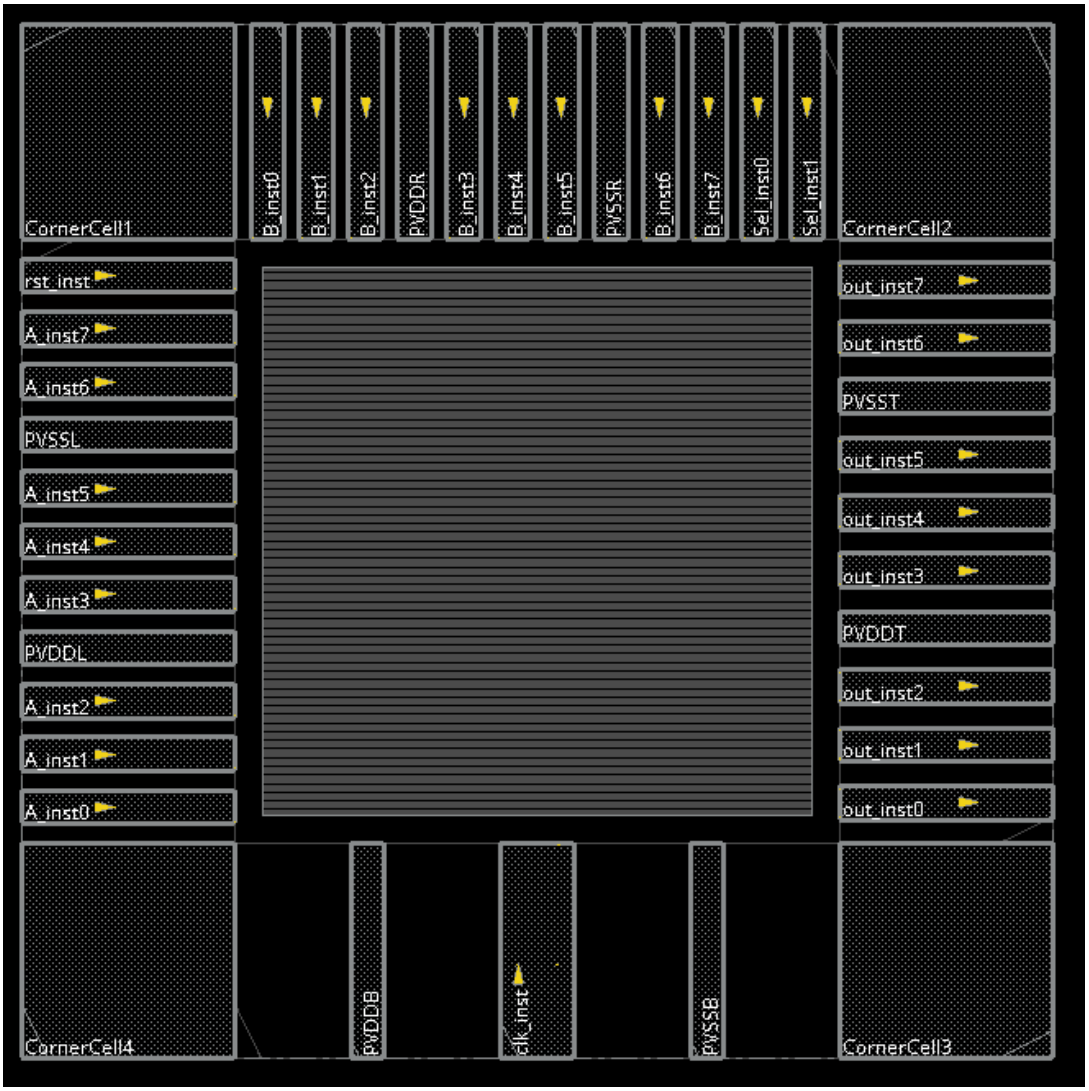
  Instance  Module  Cell Count  Cell Area  Net Area   Total Area      Wireload
--------------------------------------------------------------------------------
top_wrapper                72 166390.760     0.000   166390.760 ZeroWireload (S)
  (S) = wireload was automatically selected
```

Total area = 166382.120µm2.

To find out Required core area, use formula:

Required core area = 166382.120/0.7 ≈ 237688.74 µm2

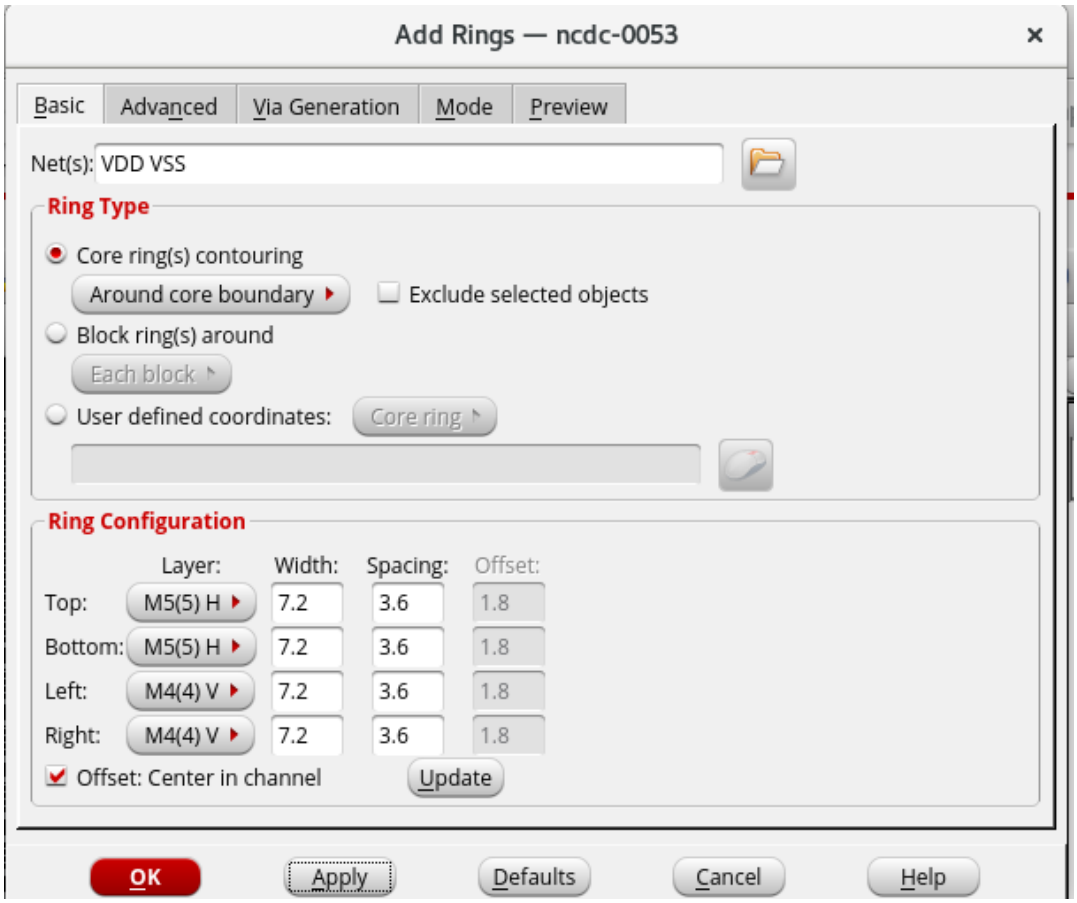
Width and height of core area = sqrt 237688.74= 487 µm



Power Planning

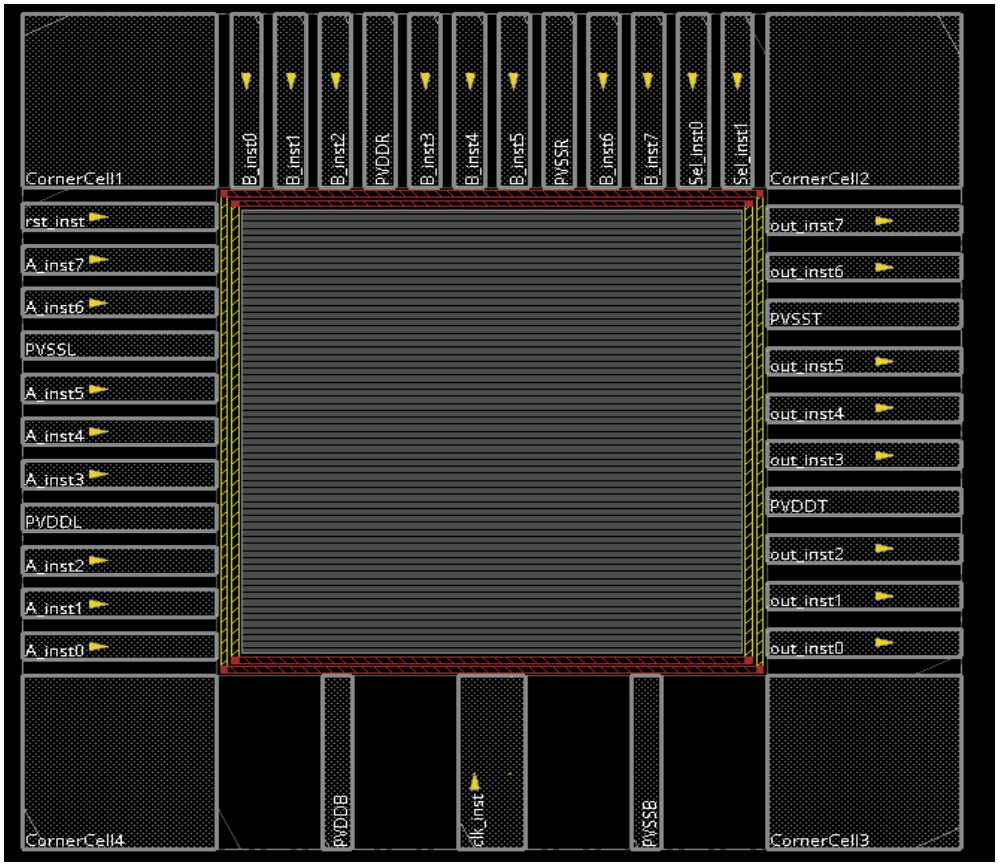
Power Rings
By giving the required parameters which we initially set while doing floorplanning & also you have done the process of this planning, the layout is look like this which is shown below:

Adding Rings:

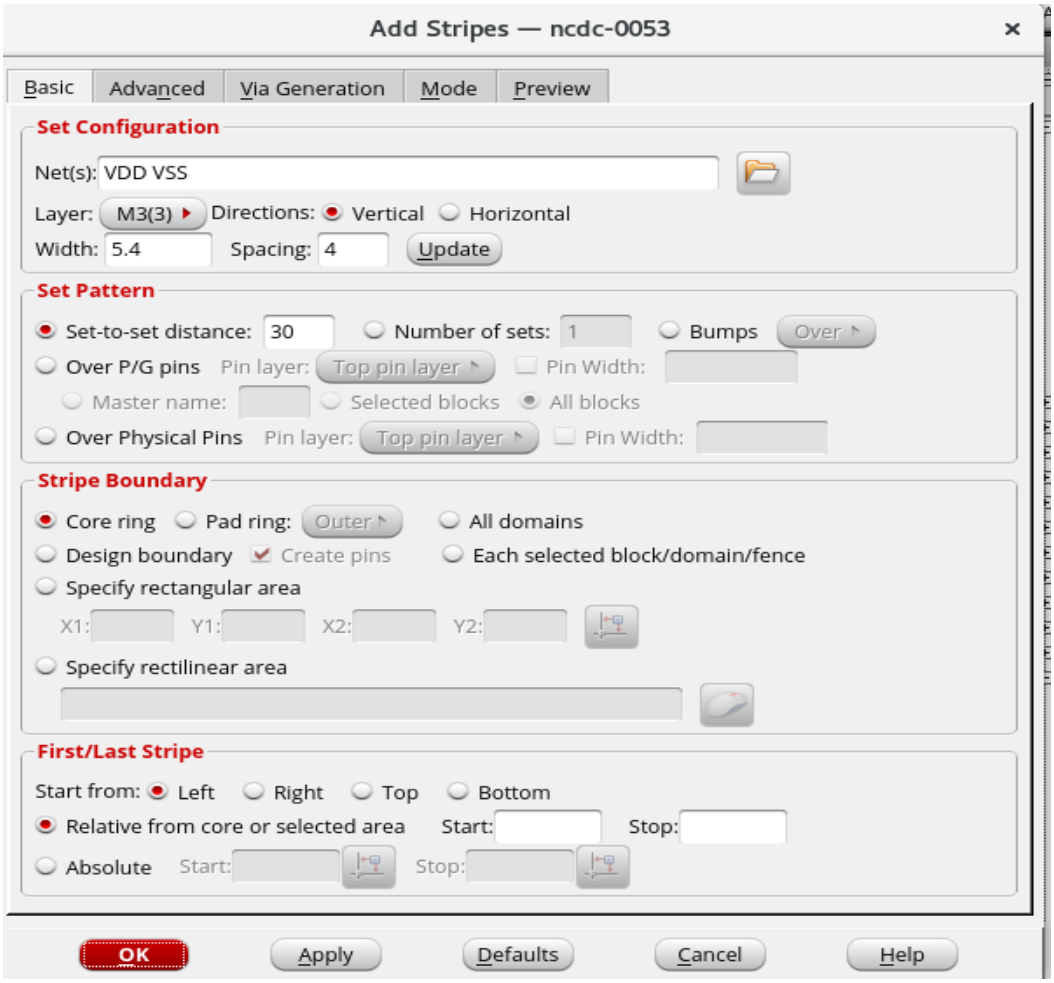


For my design i have kept

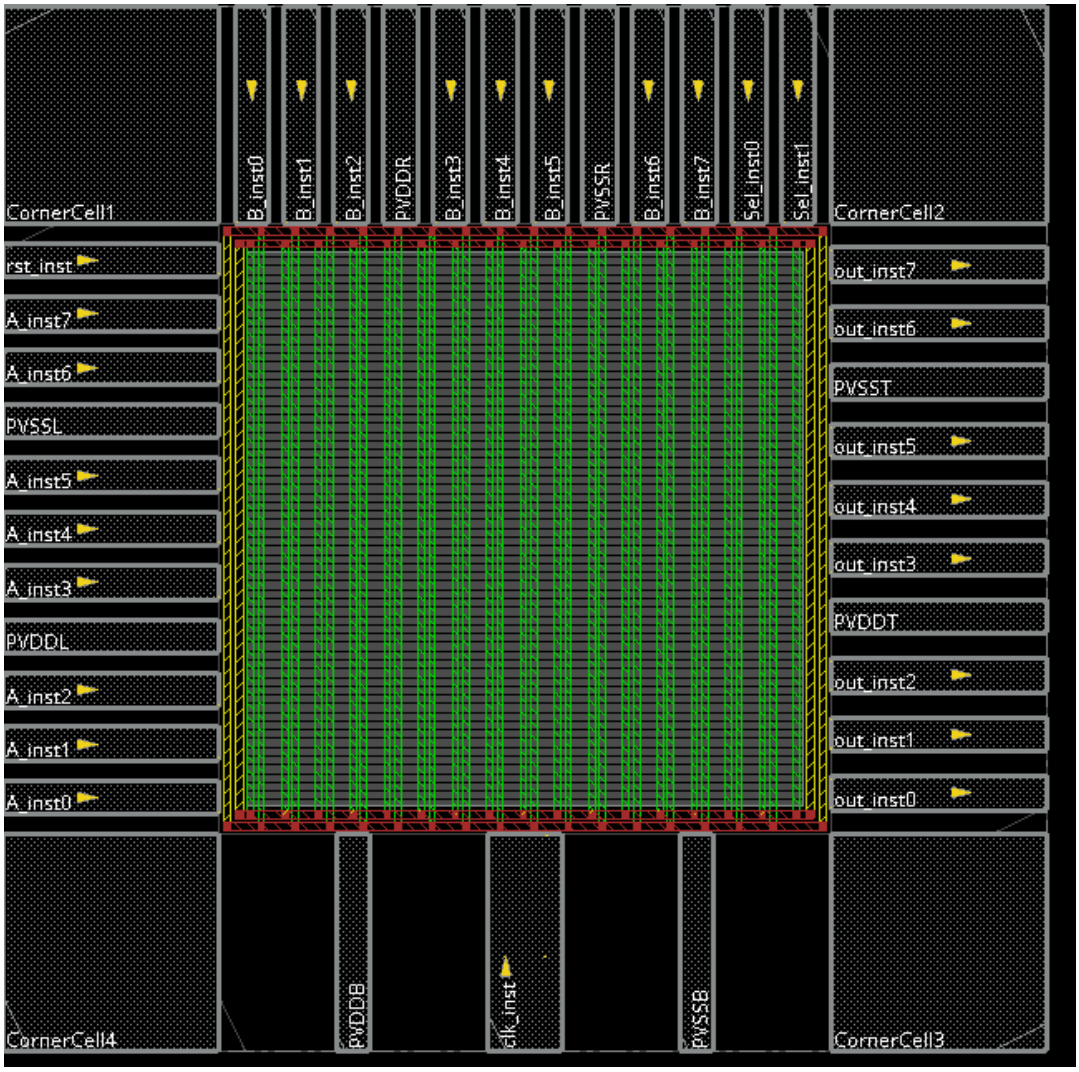
- Power ring width = 7.2 µm
- Spacing between VDD and GND = 3.6 µm
- Core to IO boundary = 7.2 (VDD) + 2 +7.2GND) = 16.4 µm of space around the core
- Offset for power rails = 25-16.4 = 8.6µm.



Adding Strips:

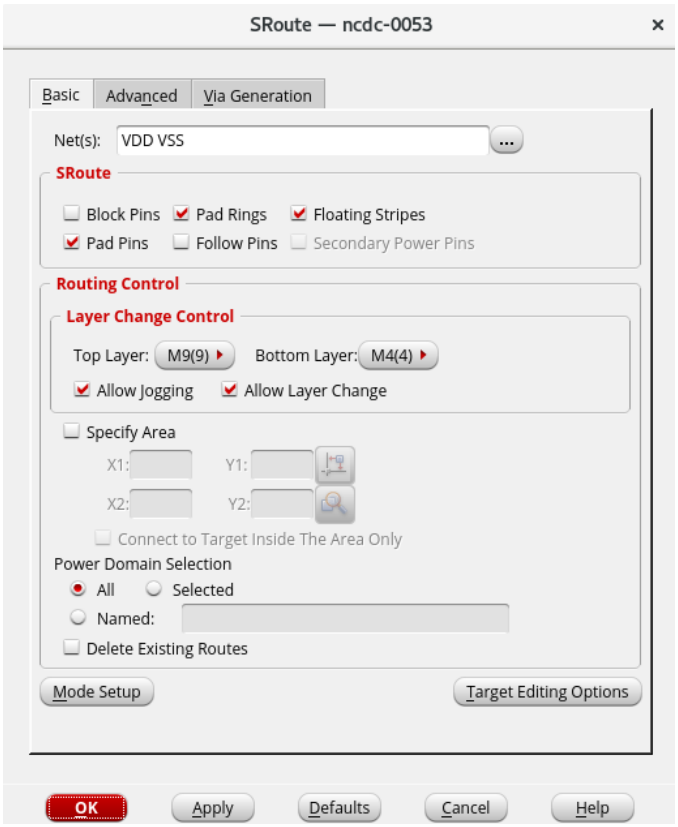


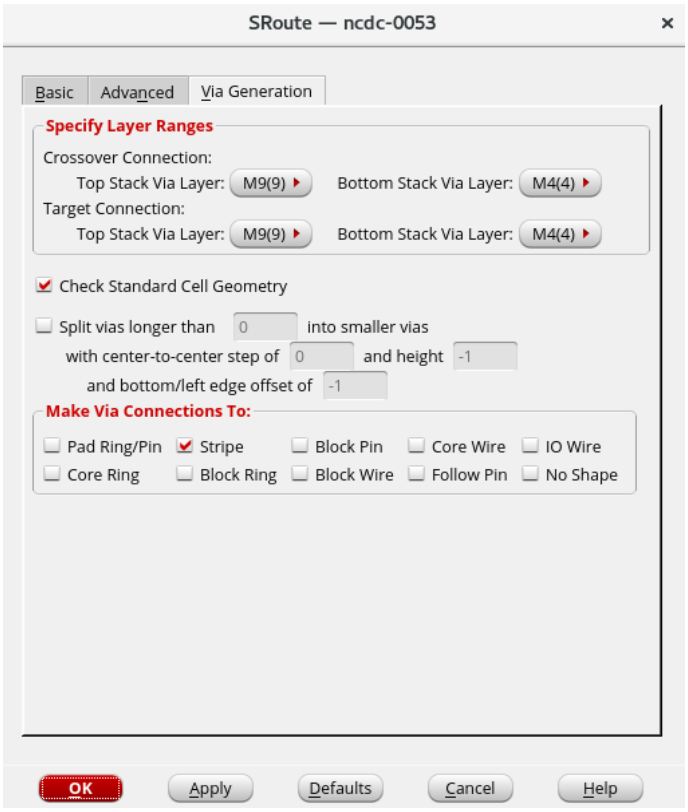
After adding strips.



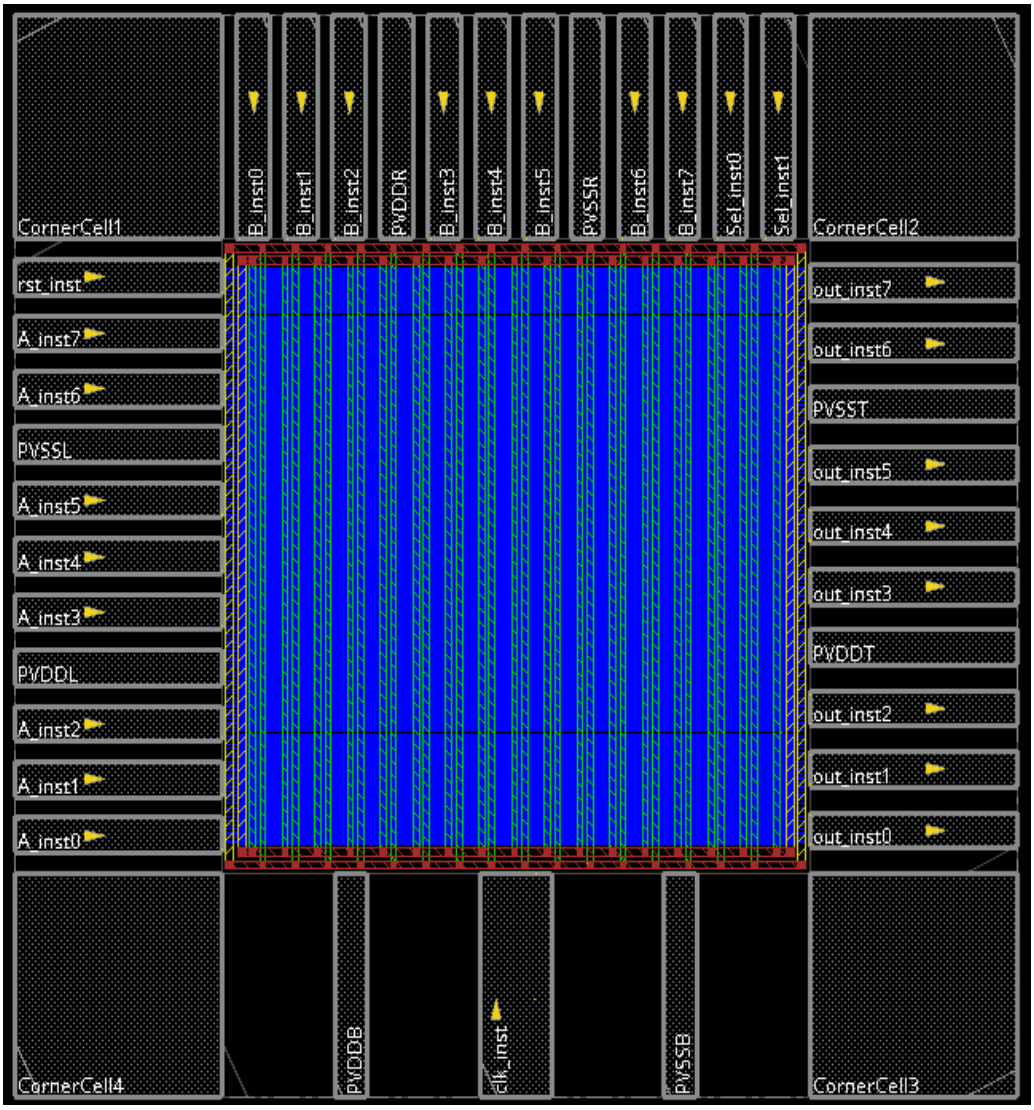
Special Route & Setting Global Nets

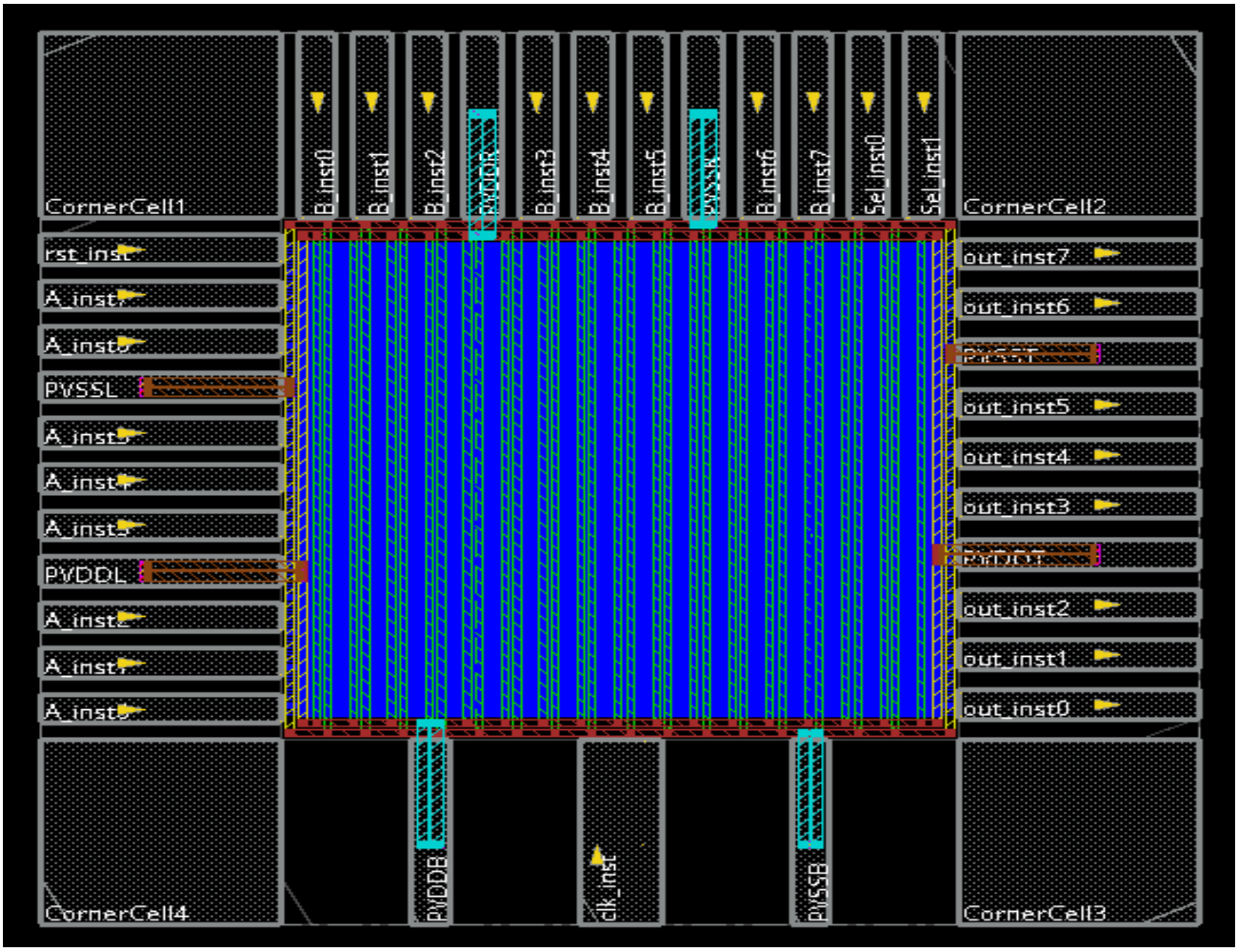
In this case, you need to **SRoute 2 times**. In 1st run, you do same as you have done in your previous labs but in 2nd run, you need to modify the specifications which is given below:



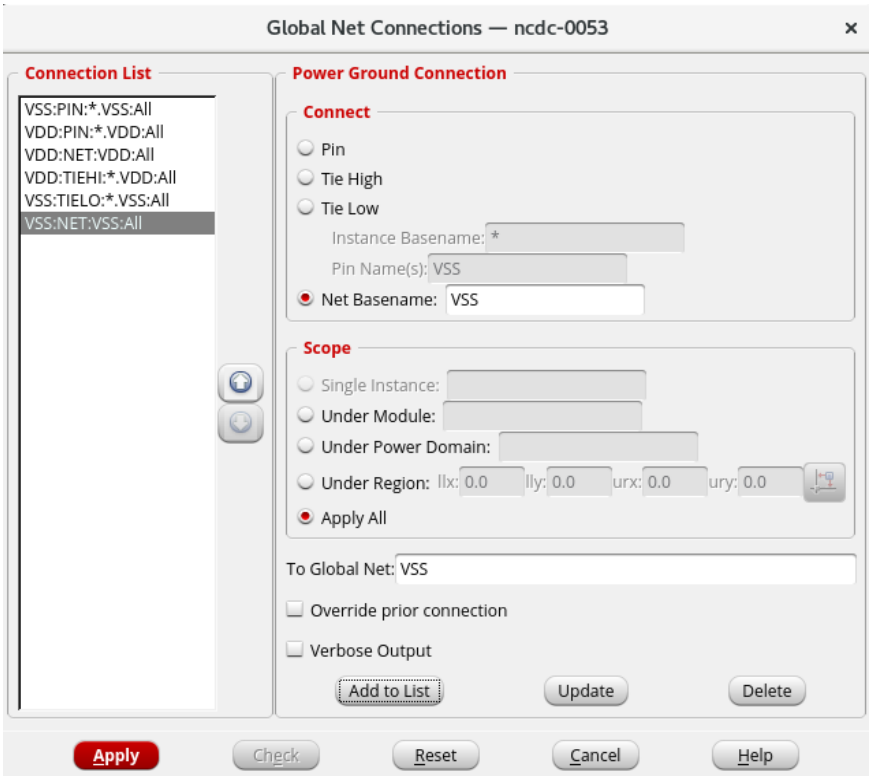


After Power Routing:



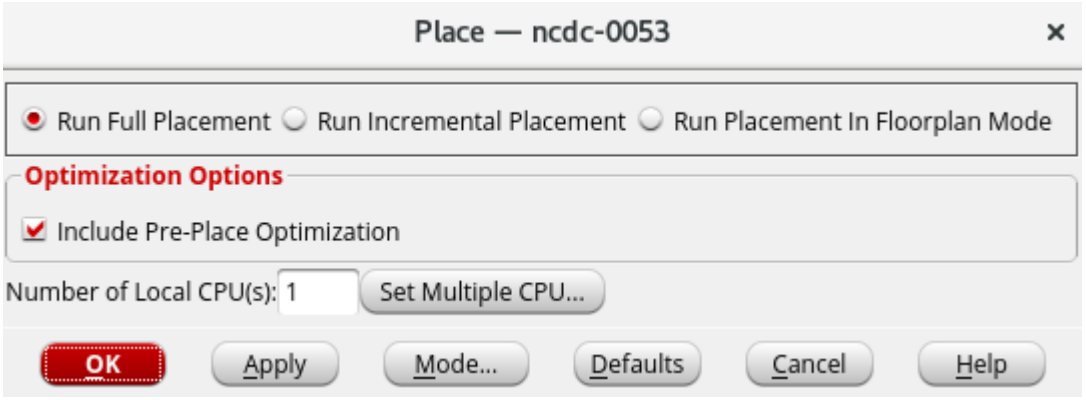

CornerCell1
B_inst0
B_inst1
B_inst2
PVDDR
B_inst3
B_inst4
B_inst5
PVSSR
B_inst6
B_inst7
Sel_inst0
Sel_inst1
CornerCell2
rst_inst
A_inst
A_inst
PVSSL
A_inst
A_inst
A_inst
PVDDL
A_inst
A_inst
A_inst
out_inst7
out_inst6
out_inst5
out_inst4
out_inst3
out_inst2
out_inst1
out_inst0
PVDDB
clk_inst
PVSSB
CornerCell4
CornerCell3

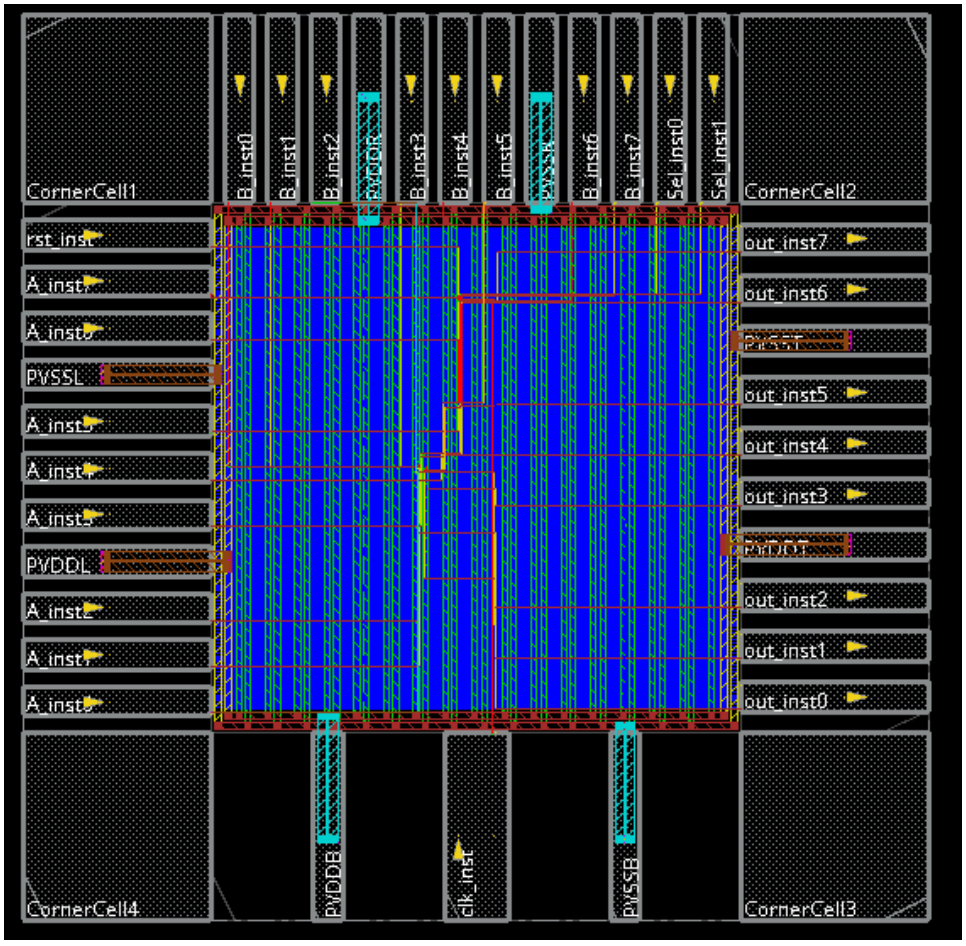
Global Nets:


Global Net Connections — ncdc-0053
Connection List
VSS:PIN:*.VSS:All
VDD:PIN:*.VDD:All
VDD:NET:VDD:All
VDD:TIEHI:*.VDD:All
VSS:TIELO:*.VSS:All
VSS:NET:VSS:All
Power Ground Connection
Connect
Pin
Tie High
Tie Low
Instance Basename: *
Pin Name(s): VSS
Net Basename: VSS
Scope
Single Instance:
Under Module:
Under Power Domain:
Under Region: llx: 0.0 lly: 0.0 urx: 0.0 ury: 0.0
Apply All
To Global Net: VSS
Override prior connection
Verbose Output
Add to List
Update
Delete
Apply
Check
Reset
Cancel
Help

Placement of Standard Cell

B giving the parameters you have done in your previous labs, the layout is look like this which is shown below:





Clock Tree Synthesis:

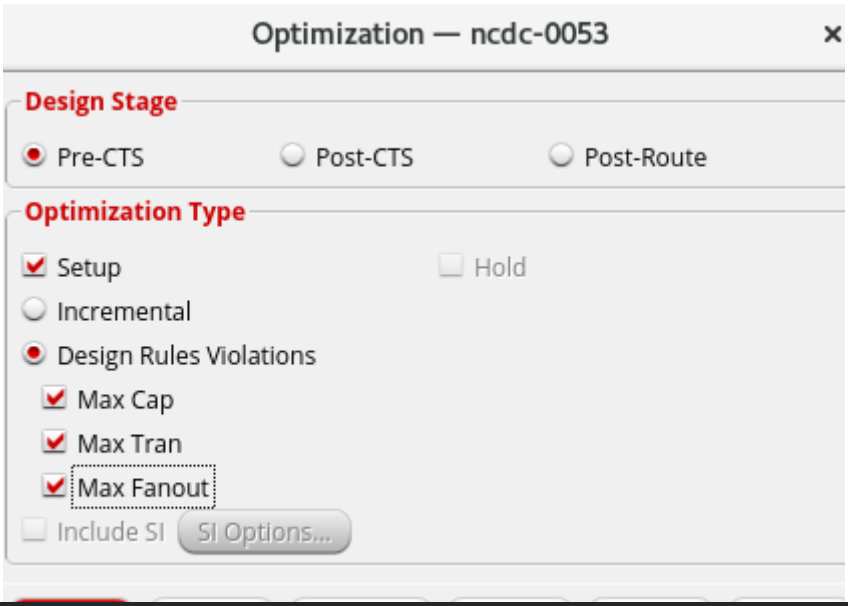
Clock Tree Synthesis (CTS) is a crucial stage in the ASIC backend flow that ensures the clock signal reaches all sequential elements (flip-flops, latches) with minimal skew and optimal delay.

First, we will perform Pre-CTS:

Its main purpose is to prepare the design for successful CTS and post-CTS timing closure.

- Fix setup timing violations on data paths.
- Prepare for clock tree balancing
- Reduce congestion and improve cell placement
- Minimize clock skew impact
- Add buffers to improve transition, fanout and capacitance.
- Legalize placement of cells to ensure no overlaps and DRC errors.

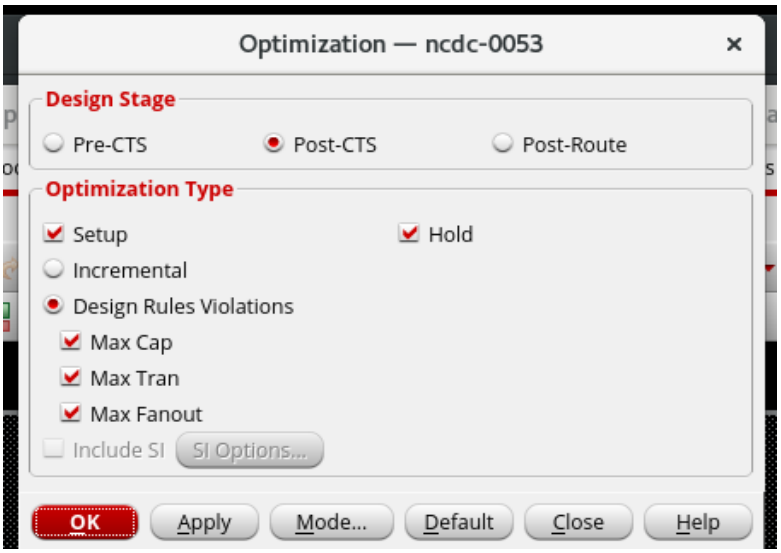
For this I go to ECO -> Optimize design.
Check Max Fanout.




```
Setup views included:
 slow

+--------------------+---------+---------+---------+
|     Setup mode     |   all   | reg2reg | default |
+--------------------+---------+---------+---------+
|           WNS (ns):|  3.973  |   N/A   |  3.973  |
|           TNS (ns):|  0.000  |   N/A   |  0.000  |
|    Violating Paths:|    0    |   N/A   |    0    |
|          All Paths:|   16    |   N/A   |   16    |
+--------------------+---------+---------+---------+

+----------------+-----------------------------------+------------------+
|                |              Real                 |       Total      |
|    DRVs        +------------------+----------------+------------------|
|                |  Nr nets(terms)  | Worst Vio      |  Nr nets(terms)  |
+----------------+------------------+----------------+------------------+
|   max_cap      |      0 (0)       |   0.000        |      0 (0)       |
|   max_tran     |      0 (0)       |   0.000        |      0 (0)       |
|   max_fanout   |      0 (0)       |     0          |      0 (0)       |
|   max_length   |      0 (0)       |     0          |      0 (0)       |
+----------------+------------------+----------------+------------------+
```




```
Setup views included:
 slow
Hold views included:
 fast

+--------------------+---------+---------+---------+
|     Setup mode     |   all   | reg2reg | default |
+--------------------+---------+---------+---------+
|           WNS (ns):|  4.119  |   N/A   |  4.119  |
|           TNS (ns):|  0.000  |   N/A   |  0.000  |
|    Violating Paths:|    0    |   N/A   |    0    |
|          All Paths:|   16    |   N/A   |   16    |
+--------------------+---------+---------+---------+

+--------------------+---------+---------+---------+
|     Hold mode      |   all   | reg2reg | default |
+--------------------+---------+---------+---------+
|           WNS (ns):|  0.256  |   N/A   |  0.256  |
|           TNS (ns):|  0.000  |   N/A   |  0.000  |
|    Violating Paths:|    0    |   N/A   |    0    |
|          All Paths:|   16    |   N/A   |   16    |
+--------------------+---------+---------+---------+

+----------------+-----------------------------------+-------------------+
|                |               Real                |       Total       |
|    DRVs        +------------------+----------------+-------------------|
|                |  Nr nets(terms)  | Worst Vio      |  Nr nets(terms)   |
+----------------+------------------+----------------+-------------------+
|   max_cap      |      0 (0)       |   0.000        |      0 (0)        |
|   max_tran     |      0 (0)       |   0.000        |      0 (0)        |
|   max_fanout   |      0 (0)       |     0          |      0 (0)        |
|   max_length   |      0 (0)       |     0          |      0 (0)        |
+----------------+------------------+----------------+-------------------+
```

After going through these processes which you have done in your previous labs, the final layout is look like this which is shown below:

```
#Up-Via Summary (total 395):
#                   single-cut          multi-cut      Total
#-----------------------------------------------------------
# M1                288 ( 99.7%)          1 (  0.3%)        289
# M2                 69 (100.0%)          0 (  0.0%)         69
# M3                 25 (100.0%)          0 (  0.0%)         25
# M4                 12 (100.0%)          0 (  0.0%)         12
#-----------------------------------------------------------
#                   394 ( 99.7%)          1 (  0.3%)        395
#
```

```
*** Message Summary: 8 warning(s), 0 error(s)
```

```
*SPEF "IEEE 1481-1998"
*DESIGN "top_wrapper"
*DATE "Thu Oct  9 13:13:53 2025"
*VENDOR "Cadence Design Systems, Inc."
*PROGRAM "Innovus"
*VERSION "21.18-s099_1"
*DESIGN_FLOW "PIN_CAP NONE" "NAME_SCOPE LOCAL"
*DIVIDER /
*DELIMITER :
*BUS_DELIMITER []
*T_UNIT 1 NS
*C_UNIT 1 PF
*R_UNIT 1 OHM
*L_UNIT 1 HENRY

// MMMC spef file for corner 'RC_WC'

*NAME_MAP

*1 CTS_1
*2 FE_OFN5_wOut_0
*3 FE_OFN4_n_35
*4 FE_OFN3_n_31
*5 FE_OFN2_n_30
*6 FE_OFN0_n_10
*7 A[7]
*8 A[6]
*9 A[5]
*10 A[4]
```

```
Path 10: MET Setup Check with Pin uut_Out_reg[2]/CP
Endpoint:   uut_Out_reg[2]/D (^) checked with  leading edge of 'CLK'
Beginpoint: rst_inst/C       (^) triggered by  leading edge of 'CLK'
Path Groups: {CLK}
Analysis View: slow
Other End Arrival Time          0.071
- Setup                         0.077
+ Phase Shift                  10.000
- Uncertainty                   0.200
= Required Time                 9.794
- Arrival Time                  4.131
= Slack Time                    5.663
     Clock Rise Edge                     0.000
     + Input Delay                       3.000
     = Beginpoint Arrival Time           3.000
     Timing Path:
     +-------------------------------------------------------------------------+
     |    Instance    |     Arc      |   Cell   | Delay | Arrival | Required |
     |                |              |          |       |  Time   |   Time   |
     |----------------+--------------+----------+-------+---------+----------|
     | rst_inst       | C ^          |          |       |   3.000 |    8.663 |
     | g1001__3680    | A1 ^ -> ZN ^ | INR2D2   | 0.335 |   3.335 |    8.998 |
     | g990__1666     | A1 ^ -> ZN v | ND2D2    | 0.345 |   3.680 |    9.342 |
     | g971__5526     | A1 v -> ZN ^ | OAI221D0 | 0.192 |   3.872 |    9.535 |
     | FE_OFC2_n_30   | I ^ -> Z ^   | CKBD0    | 0.258 |   4.130 |    9.793 |
     | uut_Out_reg[2] | D ^          | DFQD1    | 0.001 |   4.131 |    9.794 |
     +-------------------------------------------------------------------------+
     Clock Rise Edge                     0.000
     = Beginpoint Arrival Time           0.000
     Other End Path:
     +-------------------------------------------------------------------------+
     |      Instance       |    Arc     |  Cell  | Delay | Arrival | Required |
     |                     |            |        |       |  Time   |   Time   |
     |---------------------+------------+--------+-------+---------+----------|
     | clk_inst            | XC ^       |        |       |   0.000 |   -5.663 |
     | CTS_ccl_a_buf_00001 | I ^ -> Z ^ | CKBD16 | 0.070 |   0.070 |   -5.593 |
     | uut_Out_reg[2]      | CP ^       | DFQD1  | 0.001 |   0.071 |   -5.592 |
     +-------------------------------------------------------------------------+
```

DRC check:

```
 VERIFY DRC ...... Sub-Area: {42.120 376.200 125.640 459.000} 117 of 121
 VERIFY DRC ...... Sub-Area : 117 complete 0 Viols.
 VERIFY DRC ...... Sub-Area: {125.640 376.200 209.160 459.000} 118 of 121
 VERIFY DRC ...... Sub-Area : 118 complete 0 Viols.
 VERIFY DRC ...... Sub-Area: {209.160 376.200 292.680 459.000} 119 of 121
 VERIFY DRC ...... Sub-Area : 119 complete 0 Viols.
 VERIFY DRC ...... Sub-Area: {292.680 376.200 376.200 459.000} 120 of 121
 VERIFY DRC ...... Sub-Area : 120 complete 0 Viols.
 VERIFY DRC ...... Sub-Area: {376.200 376.200 459.000 459.000} 121 of 121
 VERIFY DRC ...... Sub-Area : 121 complete 0 Viols.

 Verification Complete : 0 Viols.

*** End Verify DRC (CPU: 0:00:01.2  ELAPSED TIME: 2.00  MEM: 256.1M) ***
```

The DRC check shows no violation.

Check Connectivity:

Connectivity check verifies that all pins, nets, and power/ground connections are properly connected, exactly as described in the logical netlist.

- All nets are fully connected between driver and loads.
- No floating pins or dangling nets.
- No shorts between unintended nets.
- All power (VDD) and ground (VSS) pins are tied to the appropriate rails.

Connectivity errors can cause functional failures, even if timing and DRC are clean. So I Ensures the physical layout matches the logical intent of the design.

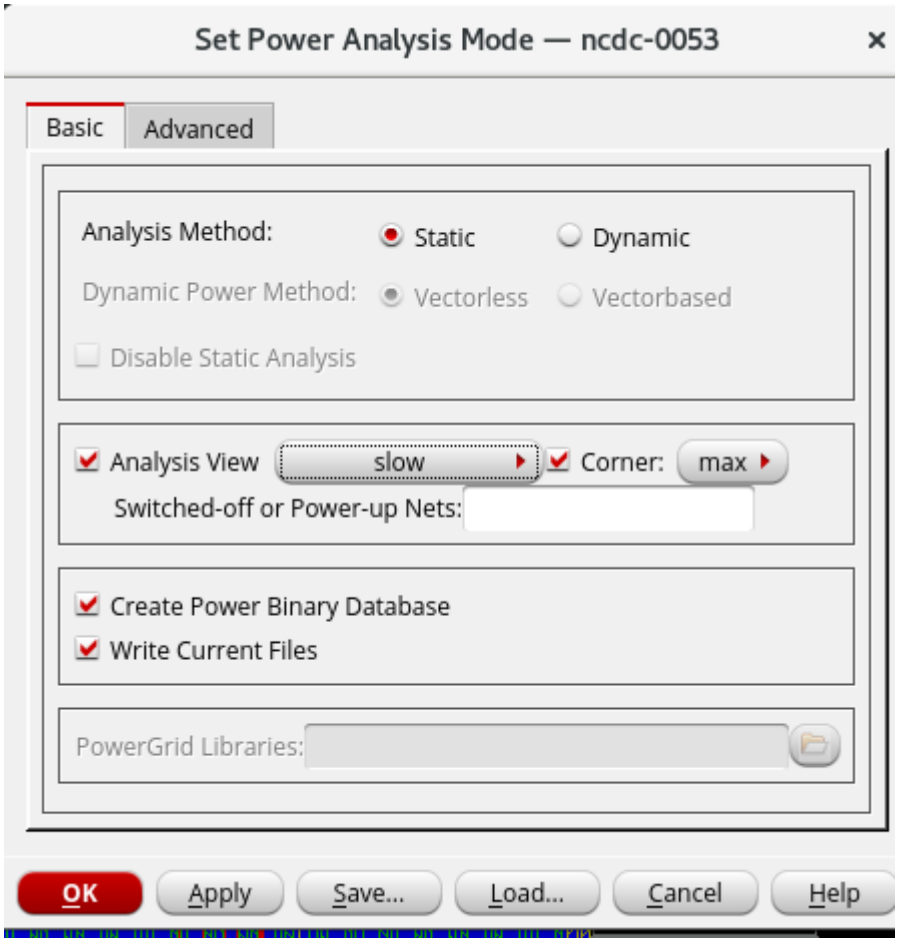
```
Design Name: top_wrapper
Database Units: 2000
Design Boundary: (-459.0000, -459.0000) (459.0000, 459.0000)
Error Limit = 1000; Warning Limit = 50
Check all nets

Begin Summary
  Found no problems or warnings.
End Summary

End Time: Thu Oct  9 13:19:44 2025
Time Elapsed: 0:00:00.0

******** End: VERIFY CONNECTIVITY ********
  Verification Complete : 0 Viols.  0 Wrngs.
  (CPU Time: 0:00:00.0  MEM: 0.000M)
```

Power Analysis:



Terminal showing power report:

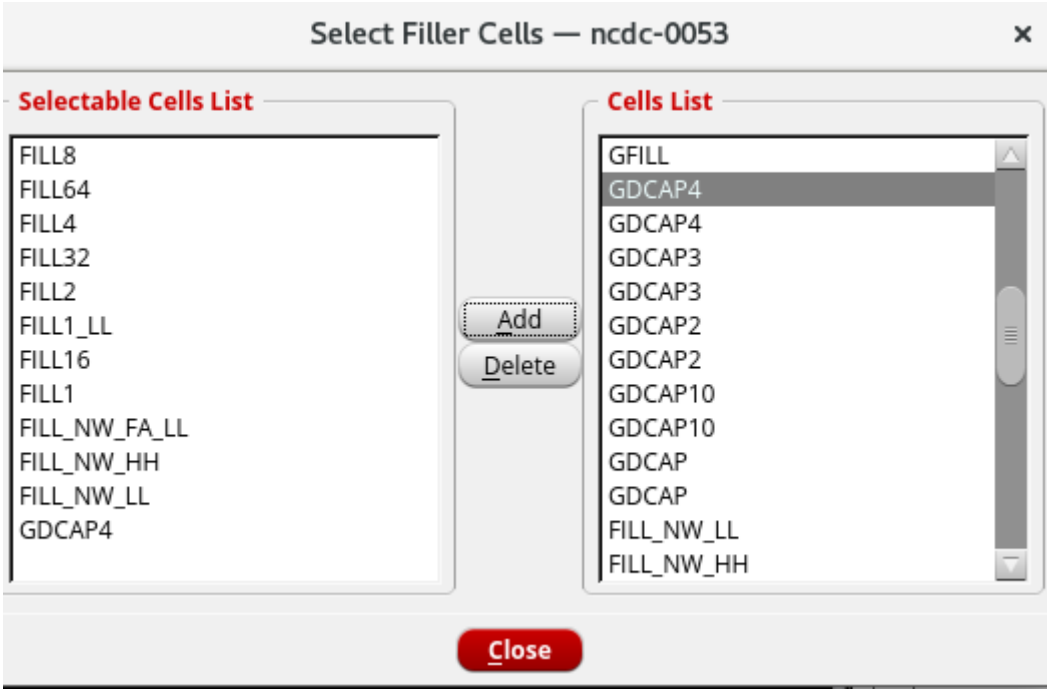
```
Total Power
-----------------------------------------------------------------------------------------
Total Internal Power:        0.01416627             0.4990%
Total Switching Power:       2.82460140            99.4994%
Total Leakage Power:         0.00004388             0.0015%
Total Power:                 2.83881155
-----------------------------------------------------------------------------------------
```

Filler Cells Placement:

Filler cell placement is the process of inserting special, non-functional standard cells (called filler cells) into the gaps between placed standard cells after place and route (P&R) is complete.

- Filler cells help maintain continuity and substrate tap spacing.
- They ensure electrical connectivity to VDD/VSS for body biasing and prevent latch-up.
- Foundries have rules requiring minimum metal density. Gaps in layout can violate these. filler cells help maintain uniform metal coverage.
- Standard cells share diffusion and poly with neighbors. If gaps are left unfilled, open diffusion regions may result in leakage or manufacturing issues.
- Without filler cells, you might get DRC errors related to well proximity, diffusion, or density violations.

Go to Place -> Physical cell -> Add filler



Terminal Output:

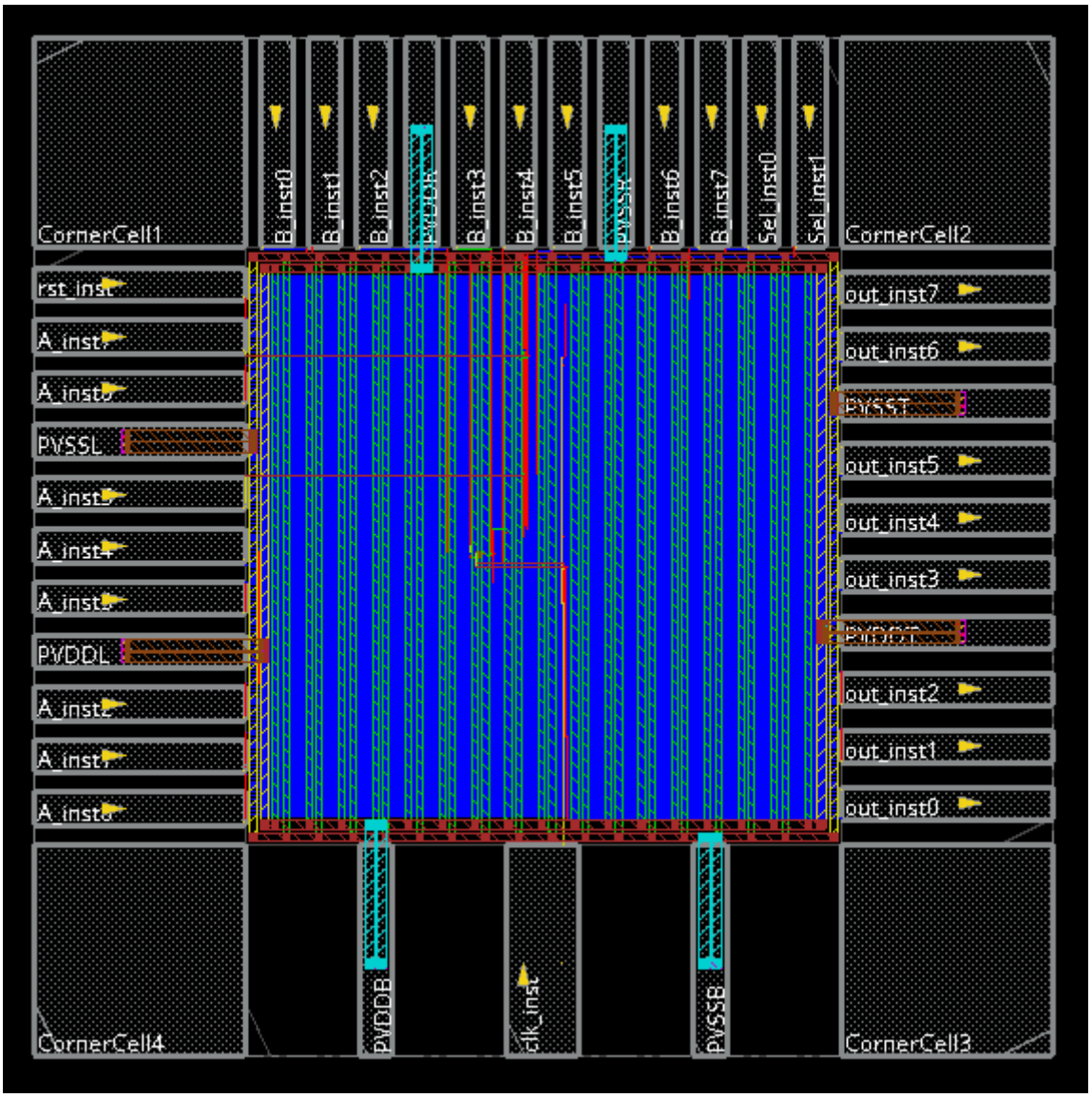
```
VERIFY DRC ..... Sub-Area: {125.640 376.200 209.160 459.000} 118 of 121
VERIFY DRC ..... Sub-Area : 118 complete 0 Viols.
VERIFY DRC ..... Sub-Area: {209.160 376.200 292.680 459.000} 119 of 121
VERIFY DRC ..... Sub-Area : 119 complete 0 Viols.
VERIFY DRC ..... Sub-Area: {292.680 376.200 376.200 459.000} 120 of 121
VERIFY DRC ..... Sub-Area : 120 complete 0 Viols.
VERIFY DRC ..... Sub-Area: {376.200 376.200 459.000 459.000} 121 of 121
VERIFY DRC ..... Sub-Area : 121 complete 0 Viols.

Verification Complete : 0 Viols.

*** End Verify DRC (CPU: 0:00:01.2  ELAPSED TIME: 1.00  MEM: 256.1M) ***
```

The final terminal shows that there is no violation in the design.

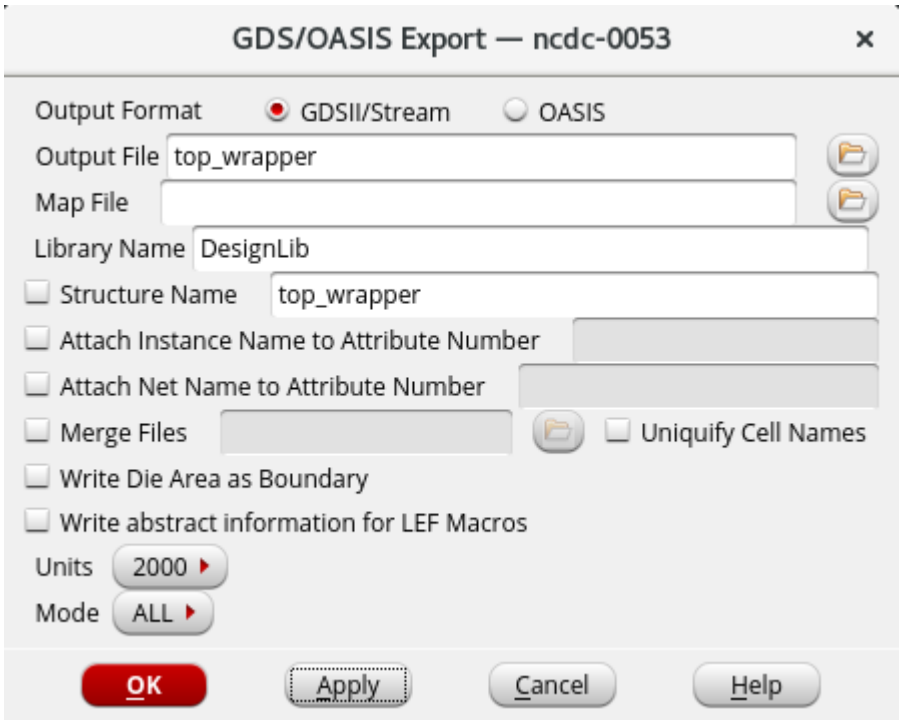
Final Design:



Generating GDSII (Graphic Data System version II):

GDSII is the final output file format used in the VLSI design flow for sending a chip layout to fabrication. It contains all the physical layout information needed by the foundry to manufacture the chip.

For this I Go to file -> save -> GDS/OASIS.



Terminal Output:

```
Nets                                763
    metal layer M1                  218
    metal layer M2                  441
    metal layer M3                   59
    metal layer M4                   37
    metal layer M5                    8

    Via Instances                   396

Special Nets                        937
    metal layer M1                  816
    metal layer M2                    8
    metal layer M3                   41
    metal layer M4                   12
    metal layer M5                   20
    metal layer M6                    8
    metal layer M7                   16
    metal layer M8                    8
    metal layer M9                    8

    Via Instances                 10844

Metal Fills                           0

    Via Instances                     0

Metal FillOPCs                        0

    Via Instances                     0

Metal FillDRCs                        0

    Via Instances                     0

Text                                100
    metal layer M1                   18
    metal layer M2                   69
    metal layer M3                    7
    metal layer M4                    5
    metal layer M5                    1


Blockages                             0


Custom Text                           0


Custom Box                            0

Trim Metal                            0

######Streamout is finished!
innovus 10>
```

Explanation:

The terminal output shows the final GDSII stream-out report generated by Cadence Innovus. This report summarizes the chip's physical design data. It lists how all metal layers (M1–M9) and via layers (VIA1–VIA8) are used for routing, and provides design statistics — such as 10,796 standard cell instances, 871 regular nets, and 937 special nets for power and clock connections. It also reports 10,844 vias connecting the metal layers and confirms that no metal fills were added. The database unit is set to 2000 DBU per micron, defining the layout's scale. The message "Streamout is finished" confirms that the GDSII file was successfully generated, representing the final chip layout ready for fabrication.

Critical Analysis:

This lab offered a complete view of the VLSI design flow — from front-end design of an Arithmetic Logic Unit (ALU) to back-end physical implementation. It covered floorplanning, placement, power planning, and routing with integrated I/O pads. At the RTL stage, functional verification ensured the ALU worked correctly before synthesis converted it into a gate-level netlist using standard cells. In floorplanning, the chip's core area was defined and I/O pads were added to connect signals and power. Placement organized the cells efficiently, while power rings and stripes were created to distribute power.

During special routing (SRoute), some issues occurred because of mismatched technology and cap-table files, which caused the VDD/VSS pads not to be recognized. This showed the importance of using consistent technology data and correct pin setup. Despite these issues, the lab clearly demonstrated how decisions made during logical design affect the physical layout. It highlighted the need for close coordination between front-end and back-end design stages to ensure that the final chip is functional, well-routed, and ready for manufacturing.